

경력기술서

가능성을 기회로 바꾸고, 제품으로 가치를 전하는
메이커 김대정입니다.

김대정

Tech Lead · Backend Engineer

marin.backend@gmail.com github.com/Dae-Jeong marinkim.xyz

경력 요약

제품·Backend·AI

기획자로 시작해 백엔드로 왔고, 아이디어를 제안한 AI 콘텐츠 제품의 FastAPI 백엔드와 AI 실행부를 직접 만들어 고객이 구독하는 서비스로 운영합니다.

이 문서는 이력서의 성과를 같은 순서로 풀어, 각 항목에서 무엇이 문제였고 무엇을 판단했으며 어떤 경계를 구현하고 어떻게 검증했는지를 적었습니다. 화면은 coding agent로 구현하고 직접 검수했으며, 수치는 측정된 것만 썼습니다.

MediSolve AI

2025.04 - 현재 · 더데이랩스
프리랜서 2025.02 - 2025.04

Tech Lead · Backend Engineer

AI 활용이 본격화되면서 모두가 메이커로 제품을 만들고 운영하는 팀에서, 제품별 결정·QA·릴리스 운영을 리드하고 Backend·AI application을 직접 구현했습니다. 팀이 같은 기준으로 만들 수 있는 Backend Template과 QA 판정 규칙은 직접 설계했습니다.

SNS 관측 데이터 · 성과 기준과 재적재 역등 평가 workflow

Threads 관측 데이터(최신 상태·시계열·원문 corpus·사람 평가)를 제품 기준과 평가 workflow로 전환

담당 범위	outcome 기준 설계 · importer-API·평가 workbench 직접 구축 · 측정 기준 재실측
문제	반응이 좋은 콘텐츠를 임계값 하나로 정의하기 어려웠고, 게시물의 최신 상태·반복 관측·원문·사람 평가가 서로 다른 생명주기를 가져 한 table로는 정합이 깨졌습니다.
선택	URL별 최신 상태와 시계열 관측을 분리하고, 절대·저자 상대·도메인 상대·참여 품질·합의의 outcome 후보를 병렬로 설계했습니다. 품질 판정은 틀리지 않았는가(자동 게이트)·플랫폼다운가(실측 분포)·통하는가(사람) 3층으로 나눠 아래층을 못 넘으면 위층을 묻지 않게 했습니다.
구현	- 기존 제품 projection과 FK를 공유하지 않는 독립 labeling bounded context(schema·migration·repository·service·API·UI workbench)를 구축했습니다. - importer는 JSONL을 typed batch로 검증하고 post는 (source, source_key) upsert, 이어쓰기는 source post 단위 replace, malformed line은 batch rollback으로 처리해 재적재를 역등하게 만들었습니다. - 사람 평가는 (source_post, labeler) unique와 upsert로 평가자별 최신 라벨만 보존하고 진행률·다음 미평가 글을 이어 볼 수 있게 했습니다.
검증	- local 격리 DB에서 전체 corpus를 두 번 적재해 건수 불변을, 증분 병합에서 기존 라벨 보존과 신규분 반영을 확인하고 API·UI·DB를 교차 대조했습니다. - 측정 표본을 소표본에서 대량 corpus로 늘리는 과정에서 프롬프트 기준값이 자사 출력을 되먹이던 순환을 재실측으로 발견해 교정했고, 반증된 접근은 로그로 남겨 재시도를 막았습니다.

결과 감에 의존하던 글쓰기 기준이 직접 수집한 실측 corpus 기반 생성·평가 기준으로 바뀌었고, 재적재·증분 병합이 정합을 깨지 않는 평가 workflow가 됐습니다.

범위 · 수집기는 별도 파이프라인이고 제 범위는 정제·적재·평가 구간입니다. 전체 corpus의 STG·Production 적재 완료는 주장하지 않습니다.

Thready Backend 재구축 · 규칙을 먼저 설계하고 돌아가는 제품을 멈추지 않고 교체

기존 frontend contract를 유지한 병렬 재구축과 단계적 전환, 이후 실사용 backend 운영 전담

담당 범위	대안 비교·설득·설계·구현·검증·전환 판단 직접 수행
문제	빠른 검증 중심으로 만들어진 초기 backend는 도메인 의존성이 얹혀 회원 로직 변경이 AI 생성 중단으로 이어졌고, 해결된 QA 이슈가 같은 영역에서 다른 형태로 재발했습니다.
선택	부분 수정을 누적하는 안과 backend만 병렬 재구축하는 안을 비교해, 서비스가 작고 AI 모듈 확장이 예정된 시점이라 재구축을 택하되 기존 frontend와 릴리스 흐름은 유지하는 범위로 한정했습니다. "돌아가는 기능을 왜 다시 만드나"에는 문제 누적 속도·AI 확장성·하네스 기반 이관 속도로 답했습니다.
구현	- 패턴·계층·검증 하네스를 먼저 세우고, 그 규칙 위에서 API·기능 inventory를 만든 뒤 새 FastAPI backend를 나란히 구현했습니다. - domain·repository·transaction 책임을 분리하고 전환 단위를 release로 관리해 frontend 호출을 단계적으로 옮겼습니다. - 재구축 범위·architecture·검증·전환 판단은 직접 소유하고, coding agent는 codebase 파악·기능 inventory·반복 구현에 썼습니다.
검증	동일 기능의 응답 비교와 QA acceptance를 통과한 범위만 전환했고, 전환 뒤에도 같은 Jira 정의로 재발을 계속 측정했습니다.
결과	같은 기준의 Jira 집계에서 해결된 QA 이슈 재오픈 비율이 37%에서 11%로, 재발 발생 일평균이 약 94% 줄었습니다(하루 4.5건에서 0.3건). 전환 이후 실제 사용자가 쓰는 backend의 배포·QA·운영을 계속 전담하고 있습니다. 범위 · 티켓에 BE/FE 라벨이 없어 제품 전체 품질 지표로 서술합니다. 초기 prototype은 다른 engineer가 만들었습니다.

주문 후속 작업과 실시간 상담 · 실패를 상태로 승격하고 순서를 보장

피부와 CRM의 주문·재고 worker(주도)와 실시간 AI 상담 runtime(공동 주 기여)

담당 범위	주문·재고 backend·worker 흐름 주도 · 실시간 상담 session·provider 경계 공동 안정화
문제	주문 생성 뒤 공급사·병원 알림이 외부 연동에서 따로 실패할 수 있었고, 실시간 상담에서는 외부 음성 모델 세션이 발화 중간에 끊기거나 보정 결과가 늦게 도착해 문맥이 바뀔 수 있었습니다.
선택	실패 가능한 작업은 API 요청 밖 worker 경계에서 처리하고 실패를 숨기지 않고 주문 상태로 승격했습니다. 실시간 상담은 세션이 끊겨도 전사가 멈추지 않도록 겹치는 세션 폴로 흡수하고, 순서는 도착 순서가 아니라 sequence가 정하게 했습니다.
구현	• 기존 task queue가 asyncio 실행 모델과 맞지 않아 교체하면서 알림 job의 PENDING → SENDING → SUCCESS/FAILED 상태, 고정 간격 재시도, 최종 실패 이력, 전용 API의 수동 재발송을 주문 상태와 연결했습니다. • 외부 음성 모델의 세션 종료 문제를 겹치는 window로 여러 세션을 시간차로 열고 세션별 버퍼와 sequence로 병합해 흡수했고, 비용 증가와 첫 응답 지연은 문서로 명시한 뒤 모델 교체 후 우회 코드를 걷어냈습니다. • DELTA·COMPLETE·CORRECTED를 같은 sequence로 묶어 늦게 도착한 보정이 다른 turn을 덮지 않게 했고, 종료 뒤 재연결되던 경합 경로의 정리 책임과 stop guard를 보강했습니다.
검증	• VAD 파라미터별 E2E 실측에서 P50 차이가 작고 주 병목이 모델 추론임을 실측해, VAD tuning 대신 DELTA 조기 trigger에 집중했습니다. • 13개 focused regression으로 회귀를 고정했고, 상담 길이 E2E에서 sequence 누락·중복 없음을 확인했습니다.

결과	API 성공 뒤 실패를 숨기지 않고 운영자가 복구 지점을 확인할 수 있는 흐름과, 외부 모델이 끊겨도 순서가 유지되는 실시간 전사 경계를 확보했습니다. 범위 · 주문 worker는 주도, 실시간 상담은 공동 주 기여입니다. 전환에 따른 성능·지연 개선 수치는 측정하지 않았습니다.
----	--

Thready AI 실행부 · 전달 보장·먹등 소비·정합 검증으로 원장과 AI 상태를 분리

AI 실행부의 독립 application·DB 분리, STG 실데이터 이관, 서비스 간 durable delivery, 생성 원장의 경합 중재

담당 범위	경계 설계·구현·migration·검증 직접 수행
문제	제품 정책·원장과 AI 생성 lifecycle이 한 backend·DB에 있어 AI 확장과 장애가 원장에 결합됐고, 분리하면 전달 유실과 역순 도착이 최신 상태를 덮을 수 있었습니다. 다중 worker가 같은 생성 원장을 두고 경합했습니다.
선택	DB를 공유하지 않고 독립 FastAPI application·DB로 분리해 인증된 HTTP 계약으로만 연결하고, 전달은 원장 변경과 같은 transaction에 기록하는 Transactional Outbox로 두었습니다. 생성 원장은 상태 전이 규칙을 entity에 복원하고 version CAS로 경합을 중재했습니다.
구현	• relay는 짧은 lease로 row를 claim하고 delivery version·attempt count를 fencing token으로 붙이며, 소비 쪽은 더 높은 version만 반영하고 stable id·natural key 충돌을 최신 row 하나로 수렴시키는 먹등 upsert/delete를 소유합니다. 최대 시도를 넘긴 전달은 terminal failure로 보존합니다. • 생성 원장은 optimistic lock으로 worker 승자를 정하고 먹등 replay 판별과 전이가 같은 판정을 쓰게 했습니다. quota는 예약 시점에 잡고 admission gate가 확정 사용량과 진행 중 예약을 함께 봐 동시 요청의 초과 실행을 막습니다. • STG 생성 이력·품질 기록·실행 추적을 parent→child 순서로 streaming copy했고 영구 cross-DB link는 쓰지 않았습니다.
검증	• local rehearsal·건수·id·status·사유를 결합한 MD5 fingerprint·FK orphan 0건으로 이관 정합성을 확인했고, stale PUT/DELETE fence test와 양쪽 서비스 전체 회귀를 통과했습니다. • health가 성공해도 생성이 실패한 사례를 계기로 배포 성공과 기능 동작을 분리한 post-deploy 생성 API E2E gate를 세웠습니다.
결과	독립 AI application·DB를 STG·Prod에서 운영 중이며, 지연·중복·역순 전달이 최신 원장 상태를 덮지 않는 복구 경계를 확보했습니다. 범위 · Prod migration 완료와 무중단 전환은 주장하지 않습니다.

조직 표준 · 팀이 같은 기준으로 만드는 FastAPI template과 evidence로 닫는 QA 판정 규칙

팀 공통 Backend 기준과 QA 판정 규칙

담당 범위	Template 설계·구축 · QA 판정 규칙 설계 · 도입 지원
문제	팀이 coding agent로 기능을 만들기 시작하면서 백엔드 경험이 적은 담당자도 구현에 참여했습니다. QA와 운영 준비 단계에서 사용량이나 동시 요청 조건에 따라 구조적인 문제가 자주 드러나 백엔드 엔지니어의 리소스가 원인 파악과 보완에 쓰였고, QA는 실행이 성공하면 통과로 보는 경우가 있어 AI 기능의 품질 판정이 흐려졌습니다.
선택	기능마다 완벽한 구조를 요구하는 대신 자주 틀리는 경계를 기본값으로 제공했습니다. Service가 transaction 정책을 선언하고 Repository는 현재 session만 resolve하게 했고, tool이 늘어나는 Agent 기능만 Hexagonal, 나머지는 MVC를 기본 구조로 두었습니다. QA는 요구사항을 REQ로 쪼개 evidence로 닫는 판정 규칙을 세워 실행 성공과 품질 통과를 분리했습니다.
구현	• @transactional(REQUIRED·REQUIRES_NEW·NESTED)·ContextVar 기반 AsyncSession resolve-owner-task guard, typed DTO 경계, naming·timezone·soft delete-cursor pagination convention, ADR·runbook·agent 작업 맥락을 template에 넣었습니다. • 전 제품 공통 Quality Evidence Harness를 QA 팀원의 서포트를 받아 작성했습니다. REQ별 evidence plan(FE·DB·Log·Network·AI quality), PASS/FAIL/UNKNOWN 판정, 반복 이슈의 자동화 승격, LLM judge 단독 승인 금지가 규칙이며, QA 팀이 운영하는 AI QA 에이전트 파이프라인의 판정 layer로 연결했습니다.
검증	• propagation·isolation·read-only·CancelledError rollback·connection cleanup을 integration test로 고정했고, 정기 개발 회의에서 패턴 선택 기준과 구조화 비용을 검토해 반영했습니다.
결과	full template으로 시작한 신규 프로그램의 STG QA에서 같은 유형의 session·connection 문제가 재관측되지 않았고, 기획·QA·디자인 담당자가 직접 구현하는 동안 백엔드는 결과 피드백과 배포 지원으로 개입을 줄였습니다. 범위 · Template과 판정 규칙의 설계는 직접 했고 각 기능 구현과 QA 에이전트 플러그인 구현은 담당자의 몫입니다. 개발 속도·결함 수의 수치는 측정하지 않았습니다.

여러 피부과의 운영·예약 Backend · server 상태가 정하는 tenant 접근 경계

외부 피부과 여러 곳의 홈페이지·관리·예약을 지원하는 multi-tenant backend monorepo (진행 중)

담당 범위	Backend architecture·Admin/Homepage API 구축 주도 · IaC 전담
문제	여러 피부과의 운영·예약을 한 backend가 지원해야 했고, client가 보내는 header가 tenant 접근 범위를 결정하던 구조가 격리의 failure mode였습니다.
선택	Admin·Homepage API를 gateway 뒤 독립 모듈로 나누고, 작업 지점을 server auth state로 소유해 권한 검증 전용 API로만 전환하되 Homepage API의 기존 계약은 유지해 변경 범위를 Admin API로 한정했습니다.
구현	• Router–Service–Repository–Model과 DI, Generic Repository, multi-tenancy·soft delete 자동 필터를 구성하고 본사 미선택 409·권한 밖 403을 구분했습니다. • Ruff·Pyright·pre-commit 정적 검증과 API·DB 설계 가이드를 구축했습니다.
검증	• shared auth middleware와 Homepage API 계약을 유지한 채 Admin API 범위에서 접근 경계를 전환했습니다. 일부 test의 skip·xfail이 남아 전체 회귀 완료는 주장하지 않습니다.

결과	tenant 접근 범위가 server 상태로 결정되는 경계로 전환하고 있습니다. 범위 · 2026-08 기준 진행 중이며 예약률·매출은 제품·팀 outcome입니다.
----	---

Memento AI

2024.10 - 2025.01

Backend Engineer

예약·결제·고객 알림 Backend를 구현했습니다. 회사 폐업으로 경력이 종료되었습니다.

예약·결제·고객 알림 Backend

FastAPI·SQLAlchemy·MySQL 기반 예약 서비스

담당 범위	예약 API·외부 결제·알림 작업 구현
문제	외부 결제 결과와 내부 예약 자산 변경이 한 번에 성공하지 않을 수 있었습니다.
선택	결제 취소·환불과 내부 상태 변경 시점을 분리해 실패 지점을 확인할 수 있게 했습니다.
구현	- 예약 API와 외부 결제를 연동하고 결제 상태에 따라 내부 자산 변경을 제어했습니다. - 다국어 알림의 즉시·예약 발송과 작업 취소·재등록·발송 이력을 구현했습니다.
검증	결제·예약·알림 상태를 분리해 실패 시 재처리 범위를 확인했습니다.
결과	예약과 결제, 후속 알림의 책임을 나눠 운영 가능한 상태 흐름을 만들었습니다.

STUDIO LAB

2021.12 - 2024.01

Vision AI Engineer → Product Manager → Backend Engineer

AI 모델에서 시작해 제품 기획과 Backend까지 역할을 넓히며 생성형 AI 커머스 제품을 만들었습니다.

SellerCanvas · 생성형 AI 커머스 제품

prototype에서 v1.0과 외부 기업 PoC까지 발전

담당 범위	PM 중심 · AI/Backend 경험을 바탕으로 제품 범위와 출시 우선순위 결정
문제	AI 기능을 데모가 아니라 실제 업무 흐름에서 쓸 수 있는 제품으로 구체화해야 했습니다.
선택	사용자 흐름과 기능 범위를 먼저 정하고 모델·Backend·화면을 제품 단위로 연결했습니다.
구현	프로토타입의 기능 범위를 정리하고 v1.0 제품과 기업 PoC로 확장했습니다.
검증	사용 시나리오와 PoC 요구를 기준으로 기능 범위와 우선순위를 조정했습니다.

결과 CES 2024 Best of Innovation AI 부문 대상 제품에 참여했습니다.

범위 · 회사·팀 제품 성과이며 개인 단독 수상으로 표현하지 않습니다.

TellingMe · 개인 프로젝트

2024.01 - 2024.12

Backend Lead · Infra

Memento AI 재직 기간과 일부 겹치는 개인 프로젝트이며 정규 경력과 분리해 기록합니다.

TellingMe · 개인 프로젝트 Backend

Spring Boot Backend와 AWS 배포

담당 범위	Backend Lead · Infra
문제	개인 프로젝트의 API와 배포 환경을 한 흐름으로 운영해야 했습니다.
선택	Spring Boot Backend와 AWS 기반 배포 구성을 함께 맡았습니다.
구현	Backend API 개발과 서비스 배포 환경을 구성했습니다.
검증	개인 프로젝트의 운영 범위에서 배포와 기본 운영을 확인했습니다.

결과 제품 Backend와 배포를 함께 맡은 개인 프로젝트 경험을 확보했습니다.

범위 · 정규 경력이 아니며 개인 프로젝트로만 표기합니다.

아이즈슬

2020.08 - 2021.06

Vision AI Engineer · Intern

Vision AI 모델과 데이터 파이프라인을 제품 기능으로 연결하는 개발·검증에
참여했습니다.

기술

Backend

Python · FastAPI · PostgreSQL · MySQL · Redis · SQLAlchemy

Async / Realtime

RabbitMQ · TaskIQ · WebSocket · SSE · STT

AI Product

LLM integration · typed prompt · structured output · evaluation · labeling

Delivery

Docker · GitHub Actions · Terraform · Azure · AWS · Kubernetes(kubeadm · Calico
· MetalLB · ArgoCD)
